PC-BASED
SMART HOME AUTOMATION SYSTEM

FINAL IMPLEMENTATION-BASED PROJECT DOCUMENTATION

Prepared by
Hailemariam Muche
Electrical and Computer Engineering

Based on the supplied Proteus schematic, MikroC firmware, and Java source code

Version 2.0 | 30 October 2026

Document Control

Field	Description
Document	PC-Based Smart Home Automation System — Final Implementation-Based Documentation
Prepared by	Hailemariam Muche
Version / Date	2.0 / 30 October 2026
Controller	PIC16F877A
Firmware	MikroC PRO for PIC
Desktop	Java Swing / jSerialComm
Simulation	Proteus Design Suite

Basis of Documentation

This revision uses the uploaded Proteus schematic, SmartHomeLCDNew.c firmware, accompanying source-text copy, and SmartHomeApp.java. Pin assignments and software behavior are taken from the supplied source. Where schematic details or component specifications are not unambiguously identified, the report flags them for verification rather than inventing values.

Contents

1. Executive Summary
2. 1. Project Introduction and Objectives
3. 2. System Architecture
4. 3. Proteus Circuit and Hardware Implementation
5. 4. Verified PIC16F877A Pin Assignment
6. 5. Temperature Measurement and Fan Automation
7. 6. Embedded Firmware Design
8. 7. Java Desktop Application
9. 8. Serial Communication Protocol
10. 9. Installation and Operating Procedure
11. 10. Verification and Test Plan
12. 11. Engineering Review: Findings and Recommended Corrections
13. 12. Limitations and Future Development
14. 13. Conclusion
15. Appendix A — Source Files Reviewed
16. Appendix B — Reproduction Checklist

Executive Summary

The PC-Based Smart Home Automation System is an integrated embedded-control and desktop application prototype. A Java Swing application running on a personal computer communicates with a PIC16F877A microcontroller over an asynchronous serial connection configured for 9600 baud, 8 data bits, no parity, and one stop bit (9600-8-N-1). The microcontroller controls three lighting channels, a fan output, and a bidirectional motor through an L293D driver. An LM35 temperature sensor is connected to the controller's AN0 input. A 16×2 character LCD provides local display of temperature and selected device-operation messages.

This edition is based on the actual Proteus schematic, the uploaded MikroC C source, and the uploaded Java Swing source. It therefore records concrete pin assignments and implemented software behavior. It also distinguishes functions visible in the design from functions that would require additional sensing or safety hardware before real-world deployment.

1. Project Introduction and Objectives

The project demonstrates PC-based supervision and control of selected household loads using a microcontroller as the hardware interface. The PC application provides device controls, serial-port selection, temperature display, and a communication log. The PIC16F877A receives one-character commands, controls output pins, reads the LM35 through its ADC, applies a temperature threshold to the fan, updates the LCD, and transmits textual feedback.

1.1 Objectives

• Develop a Java Swing application for PC-based control and monitoring.

• Interface a PIC16F877A with the PC through a serial communication link.

• Control three lamp channels independently.

• Control a bidirectional motor through an L293D driver using OPEN and CLOSE commands.

• Measure temperature using an LM35 sensor connected to AN0.

• Switch the fan ON when measured temperature is at least 30 °C and OFF below 30 °C.

• Display temperature and selected status messages on a 16×2 LCD.

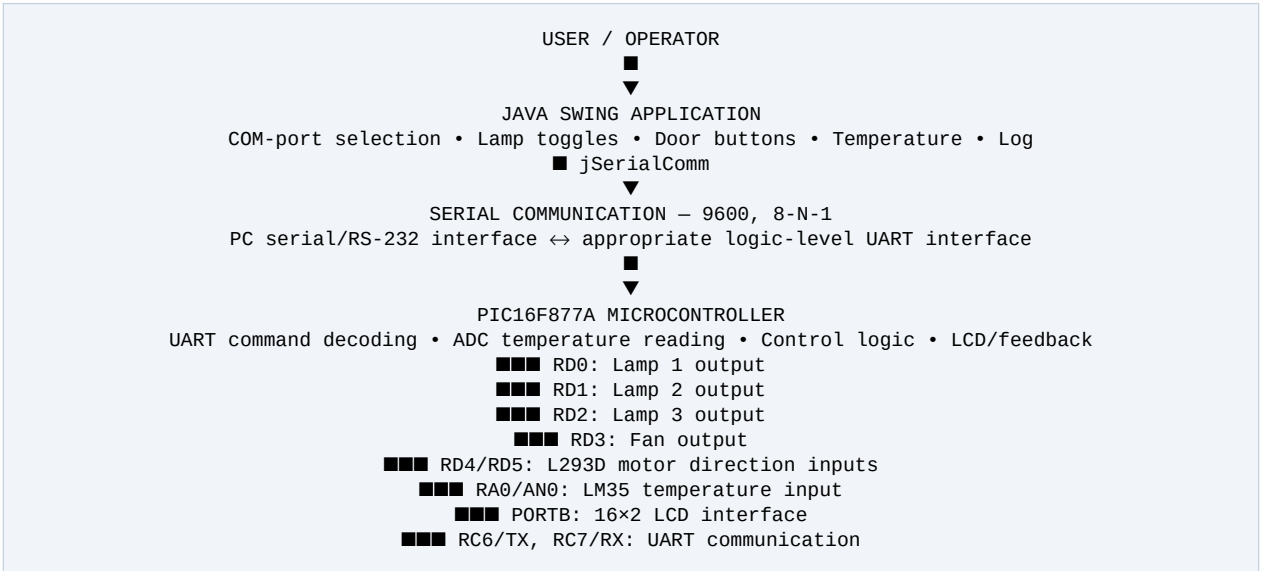
• Validate circuit behavior using Proteus simulation and hardware prototyping.

1.2 Project Scope

The documented implementation includes the PC GUI, serial communication, PIC firmware, LCD, LM35 sensor, lamp-control outputs, fan output, L293D motor interface, and Proteus circuit. The implementation is PC-connected and serial-based; wireless, cloud, and mobile-app control are not part of the supplied source.

2. System Architecture

The system has three functional layers: the PC application, the serial communication interface, and the embedded controller with its sensor/display/output circuits. The PC sends commands to the PIC16F877A. The microcontroller executes the selected action and sends status text back to the application. Independently, the firmware samples temperature and applies the automatic fan rule.



2.1 System Operation Sequence

1. The operator launches the Java application and passes the login dialog.

2. The application scans available serial ports and the operator selects the appropriate port.

3. The application opens the serial port using 9600-8-N-1 settings.

4. A lamp or door command is sent as a single character.

5. The PIC16F877A reads the received character and changes the corresponding output state.

6. The PIC sends a status string; the Java application logs received non-temperature messages and displays temperature messages.

7. The firmware repeatedly reads the LM35, updates the LCD temperature field, and switches the fan according to the 30 °C threshold.

3. Proteus Circuit and Hardware Implementation

Figure 1 is the Proteus schematic supplied with this documentation. It shows the PIC16F877A as the central controller, an LM35 temperature sensor, an LM016L 16×2 LCD, a COMPIM serial interface, relay-switched lamp loads, an L293D motor-driver IC, and associated power and protection components.

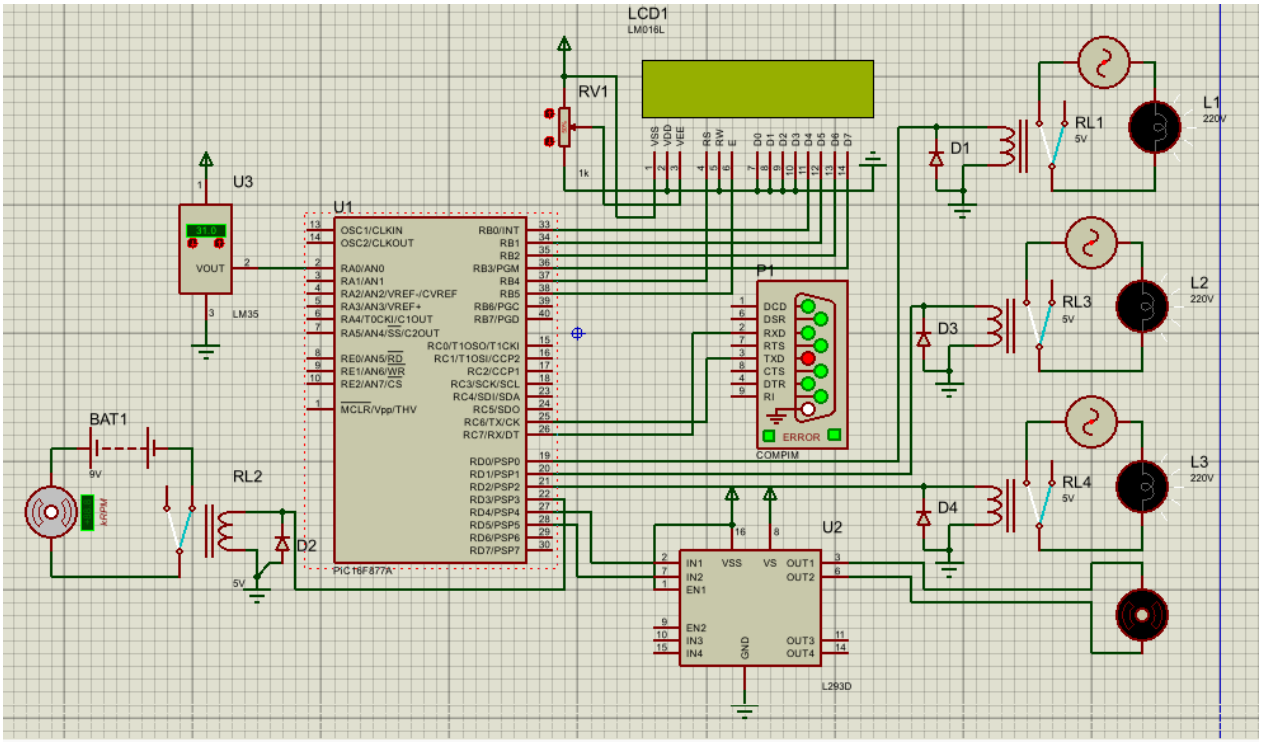


Figure 1. Proteus schematic supplied for the PC-Based Smart Home Automation System.

3.1 Main Hardware Components

Component	Function in the supplied design
PIC16F877A	Main controller for UART, ADC, LCD, and device output control.
LM35	Analog temperature sensor; output connected to RA0/AN0.
LM016L / 16×2 LCD	Local display for system heading, temperature, and selected device messages.
L293D	H-bridge motor driver; direction inputs are controlled by RD4 and RD5.
Relay stages	Switching interface for the lamp loads shown in the schematic.
COMPIM / serial interface	Proteus serial communication interface to the PC application.
Lamp loads	Three separately controlled lighting channels shown in the schematic.
DC motor	Motor load connected to the L293D output stage in the schematic.
Power sources	Provide the logic and load supplies shown in the circuit.

3.2 Schematic Interpretation

The schematic shows three 220 V lamp symbols switched through relay contacts. The PIC output signals must operate relay-driver circuitry; the microcontroller itself must not directly switch mains loads. The L293D is shown as the motor-driver stage. The uploaded firmware assigns its two direction-control signals to RD4 and RD5. The LM35 output is routed to RA0/AN0, and the LCD data/control wiring is assigned to PORTB as detailed in Section 4.

The schematic also includes an additional relay/buzzer-like branch at the left side. The supplied MikroC source does not define a corresponding output symbol or command for that branch, so this report does not assign it an undocumented firmware function.

4. Verified PIC16F877A Pin Assignment

The following assignments are taken directly from the uploaded MikroC source file SmartHomeLCDNew.c. They should be treated as the firmware's declared pin map; verify the physical net labels in the final Proteus schematic before rebuilding the circuit.

PIC port / pin	Firmware symbol	Connected function
RA0 / AN0	ADC_Read(0)	LM35 analog temperature input
RB4	LCD_RS	LCD register select
RB5	LCD_EN	LCD enable
RB0	LCD_D4	LCD data bit D4
RB1	LCD_D5	LCD data bit D5
RB2	LCD_D6	LCD data bit D6
RB3	LCD_D7	LCD data bit D7
RD0	Lamp1	Lamp 1 control output
RD1	Lamp2	Lamp 2 control output
RD2	Lamp3	Lamp 3 control output
RD3	Fan_Motor	Fan control output
RD4	Door_In1	L293D direction input 1
RD5	Door_In2	L293D direction input 2
RC6 / TX	UART1 TX	PIC-to-PC serial transmission
RC7 / RX	UART1 RX	PC-to-PIC serial reception

In the firmware, PORTD bits RD0–RD5 are configured as outputs and initially cleared. RA0 is configured as an analog input. The MikroC LCD library is initialized after ADC setup. UART1 is initialized at 9600 baud.

5. Temperature Measurement and Fan Automation

The LM35 is connected to RA0/AN0. The firmware reads ADC channel 0 with ADC_Read(0), then converts the 10-bit ADC result to temperature using the assumption of a 5 V reference and an LM35 scale factor of 10 mV per °C.

```
Temperature (°C) = ADC_value × 5.0 × 100.0 / 1023.0
```

The resulting value is formatted using FloatToStr(), trimmed using Ltrim(), and displayed on the LCD. The firmware also transmits the value to the Java application using a line beginning with TEMP: and terminated by a newline character.

Condition	Firmware action	Serial feedback
Temperature ≥ 30.0 °C	Set RD3 / Fan_Motor HIGH if it is currently LOW.	FAN_ON (on state transition)
Temperature < 30.0 °C	Set RD3 / Fan_Motor LOW if it is currently HIGH.	FAN_OFF (on state transition)

The fan state messages are transmitted when the fan changes state, not on every loop iteration. Temperature is transmitted periodically. The firmware uses a 300 ms loop delay, in addition to longer LCD/status delays in some branches.

6. Embedded Firmware Design

The uploaded firmware is written in MikroC PRO for PIC. Its main loop combines temperature sampling, automatic fan control, serial command polling, LCD status display, and periodic temperature transmission.

6.1 Initialization

• Configure RD0–RD5 as digital outputs.

• Set the lamp, fan, and both motor direction outputs LOW at startup.

• Configure RA0 as an analog input and initialize the ADC.

• Initialize the 16×2 LCD.

• Initialize UART1 at 9600 baud.

• Display the startup message SMART HOME SYS. on the LCD.

6.2 Command Processing

The firmware checks UART1_Data_Ready(), reads one byte using UART1_Read(), and processes the byte through a switch statement. Supported commands and their effects are listed in Section 8.

6.3 LCD Feedback

The LCD displays the startup title, temperature, and selected status messages such as L1_ON, L1_OFF, door movement/completion messages, and FAN_OFF. The firmware clears the LCD and uses blocking delays when displaying several status messages. This means temperature sampling and serial command servicing are temporarily paused during those delay periods.

6.4 Door Motor Timing

For command O, the firmware sets RD4=1 and RD5=0, transmits DOOR_OPENING, waits 2 seconds, then clears both direction outputs and transmits DOOR_OPENED. For command X, it sets RD4=0 and RD5=1, transmits DOOR_CLOSING, waits 2 seconds, clears both outputs, and transmits DOOR_CLOSED. The supplied source therefore implements timed motor operation rather than position feedback from door limit switches.

7. Java Desktop Application

The supplied Java source is SmartHomeApp.java. It defines a JFrame titled PC Based Smart Home Control Panel v3.0 and uses Java Swing components for the user interface. The application imports com.fazecast.jSerialComm.SerialPort for serial communication.

7.1 Main GUI Functions

• Detect and list available COM ports.

• Connect to and disconnect from the selected serial port.

• Provide toggle controls for Lamp 1, Lamp 2, and Lamp 3.

• Provide Open Front Door and Close Front Door buttons.

• Display the received temperature in a JLabel.

• Display communication and event text in a non-editable JTextArea log.

7.2 Serial Configuration and Data Handling

The application configures the selected port for 9600 baud, 8 data bits, one stop bit, and no parity. It opens the port, obtains an OutputStream for transmission, and creates a BufferedReader for incoming text. A background listener thread reads incoming lines. Lines beginning with TEMP: update the temperature label; other non-empty lines are appended to the log.

7.3 Authentication

The supplied source includes a login dialog. The credentials are hard-coded in the Java source as username admin and password password123. This is suitable only as a demonstration feature. Hard-coded credentials are not secure authentication and should be replaced before any real deployment.

8. Serial Communication Protocol

The serial interface is configured for 9600-8-N-1 at both ends. The PC transmits single ASCII command characters. The PIC transmits newline-terminated text messages.

8.1 PC-to-PIC Commands

Command	Action in firmware
A	Lamp 1 ON; RD0=1
a	Lamp 1 OFF; RD0=0
B	Lamp 2 ON; RD1=1
b	Lamp 2 OFF; RD1=0
C	Lamp 3 ON; RD2=1
c	Lamp 3 OFF; RD2=0
O	Door opening sequence; RD4=1, RD5=0 for 2 s
X	Door closing sequence; RD4=0, RD5=1 for 2 s

8.2 PIC-to-PC Messages

Message	Meaning / handling
TEMP:value	Temperature report; Java updates the temperature label.
L1_ON / L1_OFF	Lamp 1 state; Java appends the message to the log.
L2_ON / L2_OFF	Lamp 2 state; Java appends the message to the log.
L3_ON / L3_OFF	Lamp 3 state; Java appends the message to the log.
FAN_ON / FAN_OFF	Fan transition state; Java appends the message to the log.
DOOR_OPENING / DOOR_OPENED	Door opening sequence status; Java appends the message to the log.
DOOR_CLOSING / DOOR_CLOSED	Door closing sequence status; Java appends the message to the log.

The PIC source appends newline characters to each transmitted message. This is consistent with the Java BufferedReader.readLine() implementation. The Java application sends the command byte directly and does not append a newline to commands.

9. Installation and Operating Procedure

1. Build or open the Proteus project and verify component models, wiring, power supplies, and the serial COMPIM configuration.

2. Compile SmartHomeLCDNew.c in MikroC PRO for PIC using a PIC16F877A target and the correct oscillator/configuration settings for the actual circuit.

3. Program the PIC or load the generated HEX file into the Proteus PIC model.

4. Connect the PC application to the correct COM port or paired virtual serial port used by the Proteus COMPIM interface.

5. Run the Java application with the required jSerialComm library available.

6. Enter the demonstration login credentials configured in the supplied source, then select the COM port and connect.

7. Test lamp commands A/a, B/b, and C/c individually.

8. Test door commands O and X while observing the motor-driver outputs and the 2-second timed sequence.

9. Vary the LM35 temperature input and observe the LCD, fan output, and temperature message in the GUI.

10. Disconnect from the application before closing the serial connection.

The exact virtual COM-port pairing, COMPIM baud/format configuration, PIC oscillator frequency, and power-supply values should be documented from the final Proteus project settings; they are not inferred here.

10. Verification and Test Plan

The following matrix is a verification plan derived from the actual command map and source behavior. It does not claim that the tests have been executed or passed.

ID	Test	Expected result
VT-01	Open serial connection at 9600-8-N-1.	Java application opens the selected COM port and starts the listener.

ID	Test	Expected result
VT-02	Send A then a.	RD0 changes HIGH then LOW; L1_ON and L1_OFF are transmitted.
VT-03	Send B then b.	RD1 changes HIGH then LOW; L2_ON and L2_OFF are transmitted.
VT-04	Send C then c.	RD2 changes HIGH then LOW; L3_ON and L3_OFF are transmitted.
VT-05	Send O.	RD4=1/RD5=0 for approximately 2 seconds, then both LOW; door messages are transmitted.
VT-06	Send X.	RD4=0/RD5=1 for approximately 2 seconds, then both LOW; door messages are transmitted.
VT-07	Apply temperature below 30 °C.	Fan output RD3 is LOW; FAN_OFF is transmitted when transitioning from ON.
VT-08	Apply temperature at or above 30 °C.	Fan output RD3 is HIGH; FAN_ON is transmitted when transitioning from OFF.
VT-09	Observe temperature display.	LCD and Java GUI display the reported temperature value.
VT-10	Disconnect serial cable or close port.	Application remains stable; reconnection behavior is checked.

10.1 Test Evidence to Record

• Proteus simulation screenshots for each lamp command and each motor direction.

• Measured LM35 input voltage and corresponding temperature display.

• Fan output state below and above the 30 °C threshold.

• Java GUI screenshot showing COM connection, temperature, and log.

• Physical prototype photographs and test date, if hardware tests are performed.

11. Engineering Review: Findings and Recommended Corrections

Review of the supplied source reveals several implementation details that should be documented and considered before a final hardware release.

Finding from supplied files	Engineering implication / recommendation
The .c file declares char txt_temp[8], while the accompanying text copy declares char txt_temp[15].	Keep one authoritative source and size the buffer for the longest FloatToStr output plus the null terminator. Verify the MikroC library's formatting behavior.
Door movement is controlled by fixed 2-second delays; no door limit-switch input is read in the source.	The reported DOOR_OPENED/DOOR_CLOSED messages indicate completion of the timed routine, not independently verified physical position. Add limit switches and obstruction protection for a real door.
LCD and door routines contain blocking delays, including 2-second waits.	During these waits the main loop does not continuously service UART or sample temperature. Consider non-blocking state-machine timing or timer interrupts for more responsive operation.
Java toggle controls update their displayed ON/OFF state immediately when clicked, before controller feedback is checked.	The GUI may show a requested state even if the hardware is disconnected or the command fails. Drive UI state from received controller feedback and visibly distinguish requested from confirmed state.
Java receives device messages and logs them, but does not parse L1/L2/L3/FAN/DOOR messages to synchronize each control widget.	Add message parsing and update the relevant GUI state from the returned status.
The Java login uses hard-coded admin/password123 credentials.	Do not use these credentials for a deployed system. Use secure credential handling and appropriate access control.
The firmware uses a simple character protocol without documented acknowledgements, checksums, sequence numbers, or timeout recovery.	Add a defined protocol with validation, acknowledgements, timeouts, and safe behavior on communication loss.
The fan is controlled at a single 30 °C threshold.	Consider hysteresis, sensor fault detection, filtering, and calibration to reduce switching chatter and improve reliability.
The circuit image includes 220 V lamp loads.	Use isolated, correctly rated relay drivers, fusing, enclosure, insulation, and safe wiring. Never connect mains loads directly to PIC pins.
A DB-9/RS-232 connection is indicated in the system concept.	Confirm that a proper RS-232 level-shifting transceiver is present. RS-232 voltage levels must not be connected directly to PIC UART pins.

A further source consistency check is recommended before publishing the project: confirm that the MikroC .c file, the text source copy, the compiled HEX, and the Proteus PIC model all correspond to the same final firmware revision.

12. Limitations and Future Development

12.1 Current Implementation Limitations

• The system depends on a PC and a serial link.

• Door travel is timed rather than confirmed through position sensors in the supplied firmware.

• The serial protocol is simple and has limited error detection/recovery.

• The Java GUI does not fully synchronize toggle states from controller feedback.

• Authentication credentials are embedded in the source code.

• Mains switching and physical door safety require additional engineering controls.

12.2 Recommended Future Improvements

• Add door open/closed limit switches and obstruction detection.

• Replace blocking delays with non-blocking timing/state-machine logic.

• Implement serial acknowledgements, message framing, timeouts, and error detection.

• Synchronize GUI controls with controller-reported device states.

• Replace hard-coded login credentials with secure authentication.

• Add fan-control hysteresis and LM35 fault detection.

• Add a complete schematic legend, pin/net labels, power budget, and component ratings.

• Record measured test results and attach simulation/hardware evidence.

13. Conclusion

The PC-Based Smart Home Automation System integrates a Java Swing PC application with a PIC16F877A embedded controller using a 9600-8-N-1 serial interface. The supplied firmware maps RD0–RD2 to three lamp channels, RD3 to the fan, RD4–RD5 to the L293D motor direction inputs, RA0/AN0 to the LM35 temperature sensor, and PORTB to the 16×2 LCD. The system supports character-based device commands, timed motor operation, temperature reporting, and automatic fan control at a 30 °C threshold.

The uploaded source and schematic provide a concrete foundation for reproducibility and further development. The most important improvements before deployment are physical door-position/safety sensing, serial-protocol robustness, non-blocking firmware operation, secure authentication, and safe isolated switching of mains loads.

Appendix A — Source Files Reviewed

• Proteus schematic image: Smart Home Automation .png.

• MikroC source: SmartHomeLCDNew.c.

• Additional text source copy: SmartHomeSourceCodeNew (1) (1).txt.

• Java source: SmartHomeApp.java.

• MikroC project artifacts and compiled HEX were present in the firmware archive; this report does not claim a fresh compilation or hardware test.

Project repository: https://github.com/Hailemariam-Muche/PC-Based-Smart-Home-Automation-System

Appendix B — Reproduction Checklist

• PIC16F877A target and oscillator/configuration bits confirmed.

• LM35 model and ADC reference voltage confirmed.

• LCD wiring matches RB0–RB5 assignments.

• RD0–RD5 outputs match relay/fan/L293D input wiring.

• L293D supply, enable, motor supply, and current ratings checked.

• UART RC6/RC7 interface includes appropriate RS-232/logic-level conversion.

• Java dependencies, including jSerialComm, are available.

• Virtual COM/COMPIM settings match Java serial settings.

• Mains isolation and motor safety protections independently verified.

• Test results and revision identifiers recorded.